ENTERPRISE PLATFORM SPECIFICATION

QuickTask: AI-Powered Employee Task & Productivity Platform

Complete Technical Architecture, Workflows, Functional Modules, Directory Structure & Operations Manual

Project: QuickTask Full-Stack

Repository: github.com/MEHTHAB7/QUICK_TASK

Live Deployment: mehthab7.github.io/QUICK_TASK

Author: MEHTHAB7

Version: 1.0.0 (Production Verified)

Stack: React 19, FastAPI, Scikit-Learn, SQLite/Postgres

1. Purpose & Value Proposition — Why Use QuickTask?

Modern companies face high friction managing dispersed workforces across isolated applications: one tool for task assignment (Trello/Jira), another for daily attendance (HR portals), third-party chat (Slack), and manual spreadsheets for retrospective productivity calculations. This fragmentation results in delayed bottleneck identification, employee burnout, and administrative overhead. **QuickTask resolves this by consolidating all four pillars into a unified, AI-driven platform.**

> **Core Innovation:** Rather than reviewing missed deadlines weeks after they occur, QuickTask's built-in **Scikit-Learn Machine Learning Engine** actively computes employee productivity and efficiency trajectories in real-time, surfacing proactive *At-Risk Alerts* so managers can intervene before deadlines slip.

Key Strategic Advantages:

- **Unified Operational Visibility:** Centralizes task management, attendance check-ins/outs, team messaging, and analytics into one coherent dashboard.
- **Objective, Data-Driven Performance Scoring:** Removes subjective bias by calculating normalized (0–100) productivity metrics based on completion ratio, workload velocity, and attendance consistency.
- **Real-Time Instant Collaboration:** Native WebSocket duplex communication enables synchronous team discussions without external third-party chat subscriptions.
- **Automated PDF Audit Generation:** Executive summaries and productivity reports are compiled programmatically into vector PDF documents with zero external dependencies.
- **Multi-Environment Ready:** Seamless transition from lightweight local SQLite development to high-concurrency Docker Compose (Postgres + Redis) and cloud hosting.

2. System Architecture & Technology Stack

QuickTask is architected as an asynchronous, layered **Client-Server Application**. The frontend and backend communicate exclusively via authenticated JSON REST APIs and bi-directional WebSocket channels, enabling high concurrency and clean separation of concerns.

Layer	Technology	Architectural Rationale & Key Responsibilities
Frontend SPA	React 19, Tailwind CSS v4, Framer Motion, Recharts, Vite 8	High-performance responsive single-page application, 60fps animations, interactive SVG charts, client-side routing with GitHub Pages compatibility.

Backend API	Python 3.11/3.13, FastAPI, Starlette, Uvicorn, Pydantic v2	Asynchronous ASGI request pipeline, high throughput, automatic OpenAPI documentation, strict request/response data contracts.
Security & Auth	PyJWT, Bcrypt, Role-Based Access Control (RBAC)	Stateless Bearer token validation, cryptographic password salting and hashing, role authorization guards (Admin, Manager, Employee).
Database & ORM	SQLAlchemy 2.0, Alembic, SQLite (Dev), PostgreSQL (Prod)	Enterprise relational data modeling, automated schema migrations, zero-config local SQLite file persistence, pooled production queries.
AI / ML Engine	Scikit-Learn, Pandas, NumPy, Joblib	Pre-trained regression model inferring productivity scores from task completion ratio, impending deadlines, and attendance hours.
Real-Time & Docs	Starlette WebSockets, ReportLab PDF Engine	Duplex broadcasting for team messaging; programmatic vector PDF generation for executive audit summaries.

3. Directory & Codebase Structure

The repository is cleanly partitioned into modular frontend, backend, and deployment directories:

```
Quick_task/
|-- .github/workflows/deploy.yml   # GitHub Actions CI/CD to build & deploy frontend to GitHub Pages
|-- .gitignore                     # Root exclusion filter (venv, node_modules, dist, temp caches)
|-- docker-compose.yml             # Orchestration for PostgreSQL, Redis, FastAPI Backend, Frontend
|-- README.md                      # Complete installation, setup, credentials, and API docs
|-- generate_docs_pdf.py           # Programmatic ReportLab PDF documentation compiler
|-- docs/
|   +-- QuickTask_Complete_Documentation.pdf # Complete system architecture manual (this PDF)
|-- backend/
|   |-- alembic/                   # Database version scripts (migrations)
|   |-- app/
|   |   |-- api/                   # REST API routes and dependency injection (deps.py)
|   |   |   +-- endpoints/         # auth, users, tasks, attendance, analytics, reports, websockets
|   |   |-- core/                  # App settings (config.py) and security hashing/JWT (security.py)
|   |   |-- crud/                  # Data access layer (crud_user, crud_task, crud_attendance)
|   |   |-- db/                    # SQLAlchemy engine, session maker, base declarative model
|   |   |-- models/                # Database entities: User, Task, Attendance, Message
|   |   +-- schemas/               # Pydantic validation models for input/output serialization
|   |-- ml_models/                 # Scikit-learn model binary (joblib) and training pipeline
|   |-- app.db                     # Development SQLite database pre-seeded with test accounts
|   +-- requirements.txt           # Python backend dependencies
+-- frontend/
    |-- public/                    # Static assets, SVG icons, 404 fallback routing for GitHub Pages
    |-- src/
    |   |-- components/            # UI library: Sidebar navigation, metric cards, modals
    |   |-- context/               # AuthContext: JWT bearer token state & user role management
    |   |-- layouts/               # ProtectedLayout: standard dashboard framing and navigation
    |   |-- pages/                 # Views: Dashboard, TaskBoard, AttendanceLog, Analytics, Chat, Login
    |   |-- services/              # Axios HTTP client with auto-injecting JWT authorization header
    |   |-- App.jsx & main.jsx     # Client-side router configuration with base path awareness
    +-- vite.config.js             # Vite bundler configuration with Tailwind CSS v4 integration
```

4. How It Works — Core Functional Modules & Workflows

A. Role-Based Access Control & Authentication Flow

Users authenticate at /login via POST /api/v1/auth/login. The backend verifies the password hash using Bcrypt and generates a signed JWT containing the user's ID, email, and role. The frontend stores this in localStorage, which is automatically attached to every subsequent outgoing request via Axios interceptors.

- **Admin:** Global control. Manages all users, views aggregate company metrics, executes ML models, and downloads PDF reports.
- **Manager:** Departmental delegation. Assigns tasks to specific employees, monitors status boards, and reviews team attendance.
- **Employee:** Daily execution. Views assigned tasks on a personal board, updates progress, checks in/out, and chats with team members.

B. Task Lifecycle & Kanban State Machine

Tasks progress deterministically through three states: **Pending** → **In Progress** → **Completed**. Each task records priority (Low, Medium, High, Critical), assigned employee ID, deadline, and detailed description. State changes trigger updates in the database and feed into the AI productivity evaluation.

C. Attendance & Working Hours Tracking

Employees record daily check-ins upon starting work and check-outs upon completion. The backend computes active working hours, stores historical timestamps in the attendance table, and validates continuity. This prevents absentee logging and supplies critical attendance regularity metrics to the ML engine.

D. AI Productivity Prediction & At-Risk Detection

The machine learning subsystem in backend/ml_models/ utilizes a trained **Random Forest / Linear Regression Model**. It evaluates three core feature vectors:

1. **Velocity Ratio:** Ratio of completed tasks to total assigned tasks over a rolling 30-day window.
2. **Deadline Pressure:** Number of active tasks within 48 hours of expiration or overdue.
3. **Attendance Factor:** Cumulative logged hours versus scheduled working hours.

The endpoint POST /api/v1/analytics/predict-productivity outputs a calibrated score from 0 to 100. Employees scoring below 60 are automatically highlighted on the Admin dashboard with an *At-Risk Warning*, allowing managers to reassign tasks or assist the employee before performance deteriorates.

E. Real-Time Team Communication (WebSockets)

FastAPI's native WebSocket handler at /api/v1/ws/chat maintains open duplex channels with connected clients. Messages are parsed, augmented with user metadata and server timestamps, and broadcast to all active subscribers. The frontend uses React refs and hooks with auto-reconnection to provide a smooth, low-latency chat experience.

F. Automated Vector PDF Audit Reports

Admins can trigger on-demand executive summary reports via GET /api/v1/reports/productivity-report. The backend queries live database metrics, draws a clean vector layout in an in-memory byte stream using ReportLab, and streams it directly to the user's browser with an attachment disposition for instant download.

5. Complete REST API Specification

The backend adheres to RESTful conventions. Interactive documentation is available at /docs:

HTTP Method	API Endpoint	Access Role	Operation & Purpose
POST	/api/v1/auth/login	Public	Authenticates credentials; returns Bearer JWT access token.
GET	/api/v1/users/	Admin	Lists all employees, managers, and administrative users.
POST	/api/v1/users/	Admin	Registers a new employee or manager with role assignment.

GET	/api/v1/tasks/	All Roles	Fetches tasks (filtered by user assignment or admin global view).
POST	/api/v1/tasks/	Admin, Mgr	Creates and assigns a task with title, priority, and deadline.
PUT	/api/v1/tasks/{id}	All Roles	Transitions task status (Pending, In Progress, Completed).
POST	/api/v1/attendance/check-in	Employee	Logs employee daily check-in timestamp.
POST	/api/v1/attendance/check-out	Employee	Logs employee check-out and computes daily working hours.
GET	/api/v1/analytics/dashboard-stats	All Roles	Calculates system-wide counts, completion rates, and KPI trends.
POST	/api/v1/analytics/predict-productivity	Admin, Mgr	Feeds employee feature vectors into Scikit-Learn model to predict score.
GET	/api/v1/reports/productivity-report	Admin	Generates and downloads on-demand executive summary PDF report.
WS	/api/v1/ws/chat	Authenticated	Bidirectional WebSocket stream for low-latency team messaging.

6. Deployment & CI/CD Operations

QuickTask is engineered for high operational flexibility across three deployment targets:

1. Local Development (Instant Start)

- **Backend:** Run uvicorn app.main:app --reload --host 127.0.0.1 --port 8000 inside backend/. Uses SQLite (app.db).
- **Frontend:** Run npm run dev inside frontend/. Accessible at http://localhost:5173 with hot module reloading.

2. GitHub Pages & GitHub Actions (Automated Cloud CI/CD)

- **Workflow:** .github/workflows/deploy.yml triggers on pushes to main.
- **Build Process:** Node 20 environment compiles the Vite React application with production asset optimization and sets base path to /QUICK_TASK/.
- **SPA Routing Fallback:** Copies index.html to 404.html, enabling deep-linking and browser reloads without 404 errors on GitHub Pages.
- **Live URL: https://mehthab7.github.io/QUICK_TASK/**

3. Full Containerized Multi-Service Deployment (Docker Compose)

Executing docker-compose up -d --build spins up four coordinated containers:

- **frontend:** Nginx Alpine serving optimized production builds on port 80.
- **backend:** FastAPI Python 3.11 container exposed on port 8000.
- **database:** PostgreSQL container initialized on port 5432.
- **cache:** Redis in-memory cache for distributed state on port 6379.

7. Pre-Seeded Reference Accounts & Validation

The database contains pre-configured test users across all permission tiers for immediate testing:

Role	Email Address	Password	Access Level & Capabilities
Admin	admin@example.com	password123	Full administrative rights: user registration, company analytics, AI scoring, PDF export.
Manager	manager@example.com	password123	Managerial rights: task creation, employee assignment, team attendance log inspection.
Employee	employee@example.com	password123	Personal workspace: assigned tasks, drag-and-drop status update, daily check-in/out, team chat.
Employee	mehthab@gmail.com	12345678	Personal employee account: task management, time tracking, real-time collaboration.

Project Verification Sign-Off: QuickTask is fully implemented, locally verified, container-ready, and deployed to GitHub Actions. All subsystems (Authentication, Task Engine, Attendance, AI Productivity Analytics, WebSocket Chat, and PDF Reporting) have completed verification without regression.